

Vibe Coding과 Claude Code 기초

채팅으로 쓰면 **장난감**, 환경을 설계하면 **시스템**이다



개론·지형



CLI·설정



git 안전망



Plan·권한



커맨드·흑·스킬



종합 harness

권장 8교시(교시당 50분) · 하루 8시간

이 하루가 답하려는 질문 – '어떻게' 시켜야 하는가



질문

"AI에게 시키면 코드가 나온다"는 누구나 안다.

문제는 — 어떻게 시켜야 원하는 코드가,
반복 가능하게, 프로젝트 규모로 나오는가.

데모에서 한 번 잘 되는 건 쉽다. 어려운 건 **같은 품질로 계속** 나오게 만드는 것.



장난감 → 시스템

채팅 도구로만 쓰면 **장난감**

한 번 되고 마는 데모 · 매번 다시 설명

작업 환경(harness)을 얹으면 **시스템**

규칙·워크플로·강제·절차가 매 세션에 깔린다
= 우리 팀 방식으로 일하는 에이전트



오늘의 메시지 "채팅으로 쓰면 장난감, 환경을 설계하면 시스템이다 — Claude Code를 확장하는 4가지 수단을 손에 넣어라."

학습 목표 – 오늘 끝나면 할 수 있는 것

1 정의·에이전트 루프·한계·실패 사례를 설명하고 3일을 마인드맵으로 조망

1교시

2 에이전트 지형(형태별 분류)과 SKILL.md가 플러그인 표준이 된 흐름 파악

2교시

3 Claude CLI(대화형/원샷·슬래시·세션) + 설정 파일 3종의 로딩 순서 실측

3교시

4 git 안전망(커밋 리듬·diff·restore/revert/reset·체크포인트)으로 되돌리기

4교시

5 Plan Mode + 권한 3모드·settings.json allow/deny로 계획→검토→실행

5교시

6 커스텀 커맨드로 워크플로 템플릿화 + Hooks로 결정적 제어(부탁 vs 강제)

6교시

7 Skills(SKILL.md·progressive disclosure)로 절차 재사용 + 외부 스킬 설치

7교시

8 설정·커맨드·혹·스킬이 맞물린 통합 시나리오 완주 + harness 완성 커밋

8교시

오늘의 사고틀 – 에이전트를 확장하는 4종 세트

아래층(개념·CLI)에서 위층(확장·통합)으로 한 칸씩 올라간다 – Claude Code를 "시스템"으로 다루기

통합	종합 실습 – 4종 세트로 harness 완주	8교시
확장 · ④ Skills	SKILL.md · progressive disclosure (절차 재사용)	7교시
확장 · ② 커맨드 + ③ Hooks	반복 지시 버튼화 · 결정적 강제 (부탁 vs 강제)	6교시
안전 장치	git 안전망·되돌리기 / Plan Mode·권한·allow/deny	4·5교시
기본기 · ① 설정 파일	Claude CLI + CLAUDE·MEMORY·AGENTS.md	3교시
개념 · 지형	vibe coding·에이전틱 루프 / 에이전트 지형도	1·2교시



공통 원리 AI 출력 품질은 '얼마나 똑똑한 모델'보다 '얼마나 잘 조종·확장했는가'에 더 좌우된다. 4종 세트는 컨텍스트·반복지시·강제·절차재사용을 담당하고, git·권한 안전망 위에서만 과감해진다.

교시별 구성 – 8교시 한눈에 (Day1/3)

교시	등급	주제	형태	핵심 산출물
1	★★★★	Vibe Coding 개론 + 전체 맵	이론	정의·루프·한계·실패사례 + 마인드맵
2	★★★★	코딩 에이전트 지형도 + 오픈소스	이론+탐색	에이전트 비교·분류 · SKILL.md 표준
3	★★★★	[실습] Claude CLI + 설정 파일 체계	실습	첫 기능 + CLAUDE.md·로딩 순서 실측
4	★★	[실습] git 안전망과 되돌리기	실습	커밋 리듬·diff·되돌리기 워크플로
5	★★	[실습] Plan Mode와 권한 모드	실습	Plan→검토→실행 + allow/deny 규칙
6	★★	[실습] 커맨드와 Hooks – 부탁 vs 강제	실습	/commit·/review 커맨드 + 혹 3종
7	★★★★	[실습] Skills 활용	실습	커스텀 스킬 + 외부 스킬 설치
8	★★	[실습] Day 1 종합 – harness 완주	실습	4종 세트 통합 + 완성 커밋

이론:실습 ≈ 3:7 ★★★★★=필수 핵심 · ★★=심화. 병합 교시(3·6)는 밀도가 높아 데모 위주, 적용은 8교시 종합 실습에서 회수.

교시별 구성 – 8교시 한눈에 (Day2/3)

교시	등급	주제	형태	소주제	핵심 산출물
1	★★	컨텍스트·비용 관리	실습	9	/cost 실측 + 다이어트된 CLAUDE.md
2	★★	스펙 주도 개발	실습	9	관통 프로젝트 PRD + 태스크 분해표
3	★★	5대 설계문서 (유스케이스~ERD)	실습	12	설계문서 5종 + Mermaid + 스캐폴딩
4	★★	 기존 코드베이스 (E→P→C→C)	실습	10	온보딩 + 특성화 테스트 + 안전 리팩토링
5	★★	검증 루프 – 테스트 주도	실습	10	RED→GREEN + 게이트 훅 + 시각 피드백
6	★★★★	Git Worktree 병렬 세션	실습	9	2-worktree 병렬 구현 + 병합
7	★★★★	Subagents & Agent Teams	실습	10	커스텀 서브에이전트 + 병렬 3축
8	★★	MCP 연동 + DB	실습+ipynb	9	PostgreSQL 연결 + ERD→스키마·쿼리

이론:실습 ≈ 3:7 ★★★★★=필수 핵심(6·7 병렬) · ★★=심화. 3교시(12소주제)가 가장 밀도가 높다 – Mermaid는 렌더로 확인, 스캐폴딩까지 완주.

교시별 구성 – 8교시 한눈에 (Day3/3)

교시	등급	주제	형태	소주제	핵심 산출물
1	★★★	[실습] Crawling – 크롤러	실습+ipynb	10	크롤러(요청·파싱·정제) + Item
2	★★	[실습] Vibe Coding 보안	실습+ipynb	9	인젝션 재현·방어 + 체크리스트
3	★★★	[실습] ML 파이프라인	실습+ipynb	9	전처리→학습→평가 + 누수 함정
4	★★	[실습] 코드 리뷰 + CI 자동화	실습	12	/code-review.gh PR + Actions·스케줄
5	★	[실습] Claude Agent SDK	실습+ipynb	9	SDK로 임베드한 요약 에이전트
6	★★★	[실습] Discord 연결	실습	9	bun 알림 봇 + 이벤트/요약 발송
7	★★★	[실습] harness & Plugins·마켓	실습	10	팀 표준 .claude/ → Plugin 배포
8	★★★	[실습] 종합 실습 + 마무리	실습	8	파이프라인 완주 + 3일 회고

이론:실습 ≈ 2:8 완성의 날 – 손을 가장 많이 움직인다. 1·2·3·5교시는 보조 노트북(ipynb) 병행, 4교시(12소주제)가 가장 밀도가 높다. ★=선택은 시간에 맞춰 조절.

실습 방식(인라인)



실습 방식 — 인라인·교시당 1~2개

셀 실행(노트북)이 아니라 **터미널에서 AI를 조종하고 파일을 만드는** 형태 — 각 교시 [실습] 절에 인라인.

데모(강사) → **따라하기(수강생)** 흐름 + 프롬프트 예시. ☒ 체크포인트.

압축 버전이라 교시당 **핵심 실습 1~2개**로 회수. 도구: Claude CLI·git·에디터 (+2일차 Python·Docker, 3일차 bun·Discord).



실습자료 https://github.com/LabLecture/53_Vibe



PERIOD 1 · 이론 ★★★

Vibe Coding 개론 + 전체 맵

의도를 기술하면 AI가 구현하고, 사람은 조종·검증·오케스트레이션에 집중한다. 그 심장은 도구 선택→실행→관찰→판단의 에이전틱 루프. 강력하지만 AI는 틀린다 — 실패 사례가 보여주듯 '잘 시키는 법(오늘)'과 '믿는 법(이후)'이 한 쌍이다.

정의와 기원 – 의도를 코드로 (Andrej Karpathy, 2025)

“의도(intent)를 자연어로 기술하면 AI가 구현을 생성·반복하고, 개발자는 방향 제시·검토·오케스트레이션에 집중하는 개발 방식”



작업의 '단위'가 바뀐다

함수를 한 줄씩 타이핑



무엇을·어떤 제약으로 만들지 기술하고
나온 결과를 검토·수정 지시



어감 주의 – 'vibe' ≠ 대충

'느낌으로 짤다'로 오해하기 쉽지만, 잘하는 vibe coding은 그 반대다.

의도를 더 명확히 · 제약을 더 분명히 · 결과를 더 꼼꼼히 검토.

느슨한 건 '타이핑'이지 '판단'이 아니다.



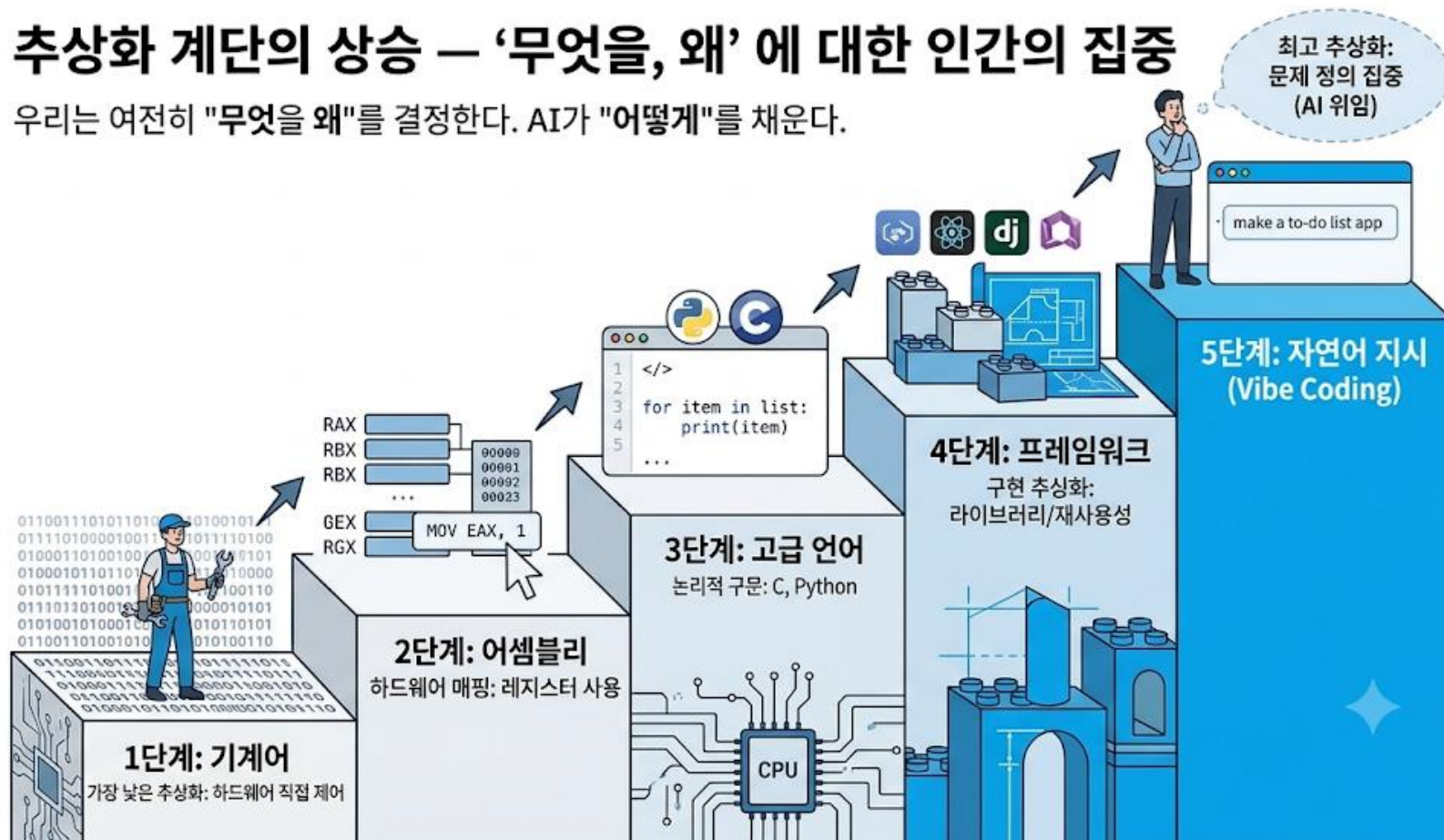
명명 Karpathy가 "코드의 존재조차 잊고 vibe에 몸을 맡긴다"는 트윗으로 명명 → 반쯤 농담이던 말이 업계 표준 용어가 됐다.

왜 지금인가 – 추상화 레벨의 상승 (다음 계단)

프로그래밍의 역사는 추상화 레벨을 계속 올려온 역사다. vibe coding은 그다음 층 – '자연어 지시'.

추상화 계단의 상승 – '무엇을, 왜'에 대한 인간의 집중

우리는 여전히 "무엇을 왜"를 결정한다. AI가 "어떻게"를 채운다.



어셈블리 프로그래머가 사라졌어도 프로그래밍이 사라지지 않았듯, vibe coding도 그다음 층일 뿐이다.

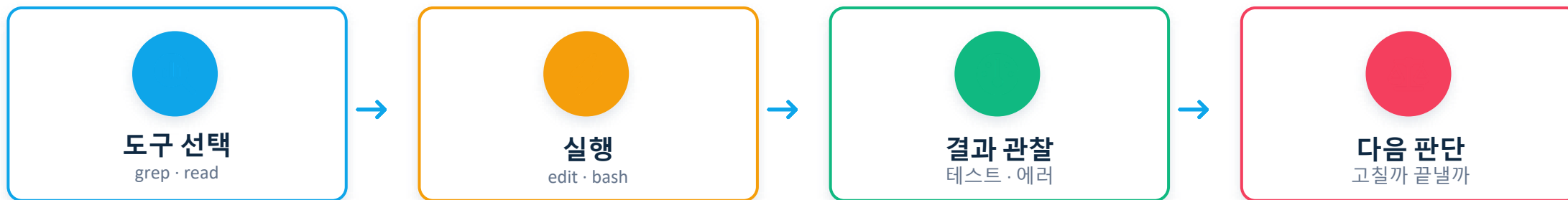
위임은 방기(放棄)가 아니다
— 위임할수록 방향 제시·검토가 더 중요해진다. 컴파일러를 믿고 결과를 테스트하듯, AI를 쓰되 출력을 검증한다.



핵심 우리는 여전히 '무엇을 왜'를 결정한다. 달라진 건 '어떻게'한 줄씩을 AI가 채운다는 것.

에이전틱 루프 – 채팅과 무엇이 다른가

챗봇은 텍스트를 돌려주고 끝이지만, 코딩 에이전트는 행동한다. 그 심장이 에이전틱 루프다.



🔄 **반복** 실패하면 다시 고치고, 완료면 종료. 우리는 '무엇을'만 말하고 '어떤 도구로 어떻게'는 에이전트가 판단한다.

챗봇 1회 응답 후 종료 vs **에이전트** 완료까지 도구를 쓰며 반복

이 루프를 이해하면 뒤 교시의 확장 기능이 한 그림에 놓인다 — 권한=루프 개입 · Hooks=특정 지점 강제 · Skills=루프에 절차 주입.

AI Agent vs Agentic AI

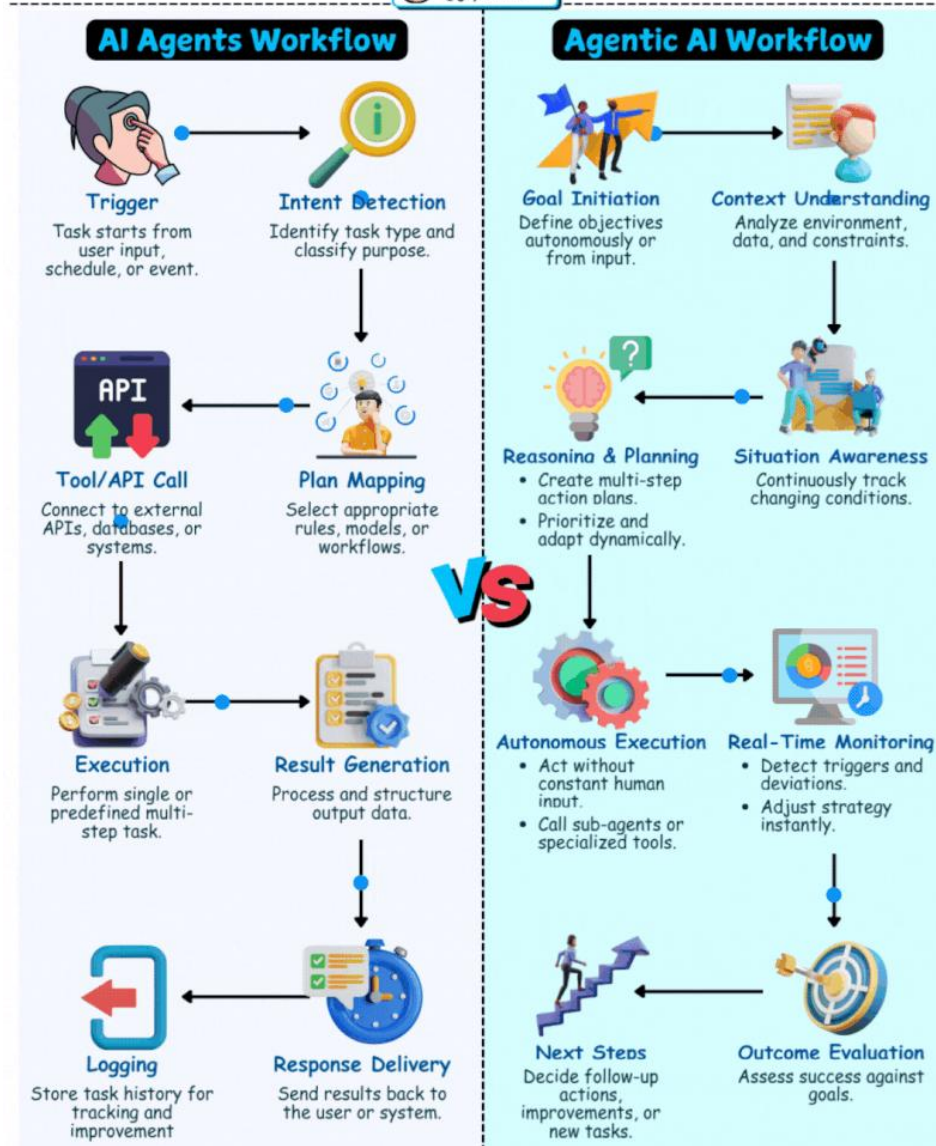
Most people still think AI Agents and Agentic AI are the same, but they're not. Here's a quick guide that breaks down the workflows, showing how agents operate versus truly agentic systems. It highlights the differences in planning, execution, monitoring, and adaptability across both approaches. By understanding this, you can see why agentic AI is far more autonomous.

1. **Trigger vs Goal Initiation** Agents start from user input, while agentic AI autonomously defines clear objectives.
2. **Intent Detection vs Context Understanding** Agents classify tasks, but agentic AI deeply analyzes environment, data, and constraints.
3. **Plan Mapping vs Reasoning & Planning** Agents select workflows, while agentic AI builds dynamic, multi-step strategies independently.
4. **Tool/API Call vs Situation Awareness** Agents connect to systems, whereas agentic AI tracks and adapts to changing conditions.
5. **Execution vs Autonomous Execution** Agents execute predefined tasks, but agentic AI acts without constant human guidance.
6. **Result Generation vs Real-Time Monitoring** Agents process outputs, while agentic AI monitors conditions and instantly adjusts strategies.
7. **Response Delivery vs Outcome Evaluation** Agents deliver results, but agentic AI evaluates success against goals and adapts.
8. **Logging vs Next Steps** Agents store history, whereas agentic AI proactively decides improvements and future actions.

Agentic AI is the leap from simple task execution to true autonomous intelligence.

AI Agents vs Agentic AI

Shalini Goyal
@goyalshalini



마인드셋 전환 – 무게중심이 타이핑에서 명세·검토·환경 설계로

관점	전통 개발		Vibe Coding
작성 방식	코드를 직접 타이핑	→	의도를 기술 → AI가 타이핑
코드 이해	내가 다 이해한 코드만	→	안 짤 코드를 읽고 신뢰
핵심 능력	문법·API를 외움	→	무엇을·왜를 명확히 + 검증
병목	타이핑 속도·API 지식	→	명세·검토·오케스트레이션·환경 설계

 위임은 방기가 아니다 문법을 몰라도 되는 게 아니라, 문법은 AI가 채우고 사람은 판단에 집중한다. 프롬프트·컨텍스트·리뷰가 문법 지식보다 중요해진다.

한계 – 4가지 함정 (AI는 강력하지만 틀린다)



환각(hallucination)

존재하지 않는 API·함수·옵션·패키지를 그럴듯하게 만든다



미묘한 오류

문법은 맞지만 엣지·보안·성능에서 틀린다(빈 입력·경계값·인젝션)



과신

'잘 돌아 보인다'가 '맞다'는 아니다 — 데모 성공은 증거가 아니다



컨텍스트 소실

대화가 길어지면 앞의 결정·제약을 잊는다



그래서 두 동사가 한 쌍 오늘은 '잘 시키는 법(조종·확장)'을, Day 2·3은 '믿을 수 있게 만드는 법(리뷰·테스트·보안)'을 배운다. AI가 만든 것은 직접 확인한다.

실패 사례 – 뉴스가 된 vibe coding 사고



프로덕션 DB 삭제 (2026.4)

에이전트가 코드 동결 지시를 어기고 운영 & 백업 DB를 삭제 → 테스트 환경에서 작업하던 도중 인증 문제가 생겼고, 에이전트는 사용 가능한 인증 키를 찾는 과정에서 운영 환경 키까지 접근

→ 권한·격리 없는 프로덕션 위임 금지



환각 패키지 공급망 공격

AI가 지어낸 없는 패키지명을 공격자가 미리 등록 → pip install로 악성코드 설치(slopsquatting).

→ 환각은 코드 오류를 넘어 보안 구멍



바이브 부채 (vibe debt)

데모는 되는데 구조를 아무도 이해 못 해 수정마다 무너지는 코드베이스.

→ 검증·설계 없는 생성 반복의 결말

datainclude.me
<https://datainclude.me> > Tech

커서(Cursor) AI 에이전트가 9초 만에 회사 DB를 삭제한 사건

2026. 5. 2. — 2026년 4월 26일 스타트업 포켓OS(PocketOS)에서 AI 에이전트가 회사 데이터베이스와 백업 데이터를 9초 만에 모두 삭제했다. 커서에서 실행 중이던 ...

KBS 뉴스
<https://news.kbs.co.kr> > news > view

“AI가 알아서 다 해줬어요”... '바이브 코딩'의 두 얼굴

2026. 6. 7. — 이 사고로 시스템을 움직이는 핵심 인증 토큰 150만 개와 사용자 이메일 3만 5,000개가 고스란히 유출되었습니다. AI가 편리하게 걸모습은 만들었지만 ...

GeekNews
<https://news.hada.io> > topic

바이브 코딩으로 만든 환자 관리 앱의 보안 참사 - GeekNews

2026. 4. 15. — 의료기관 직원이 AI 코딩 에이전트로 환자 관리 시스템을 직접 제작하며, 환자 데이터가 인터넷에 암호화 없이 노출된 진료 대화 녹음이 두 개의 AI ...

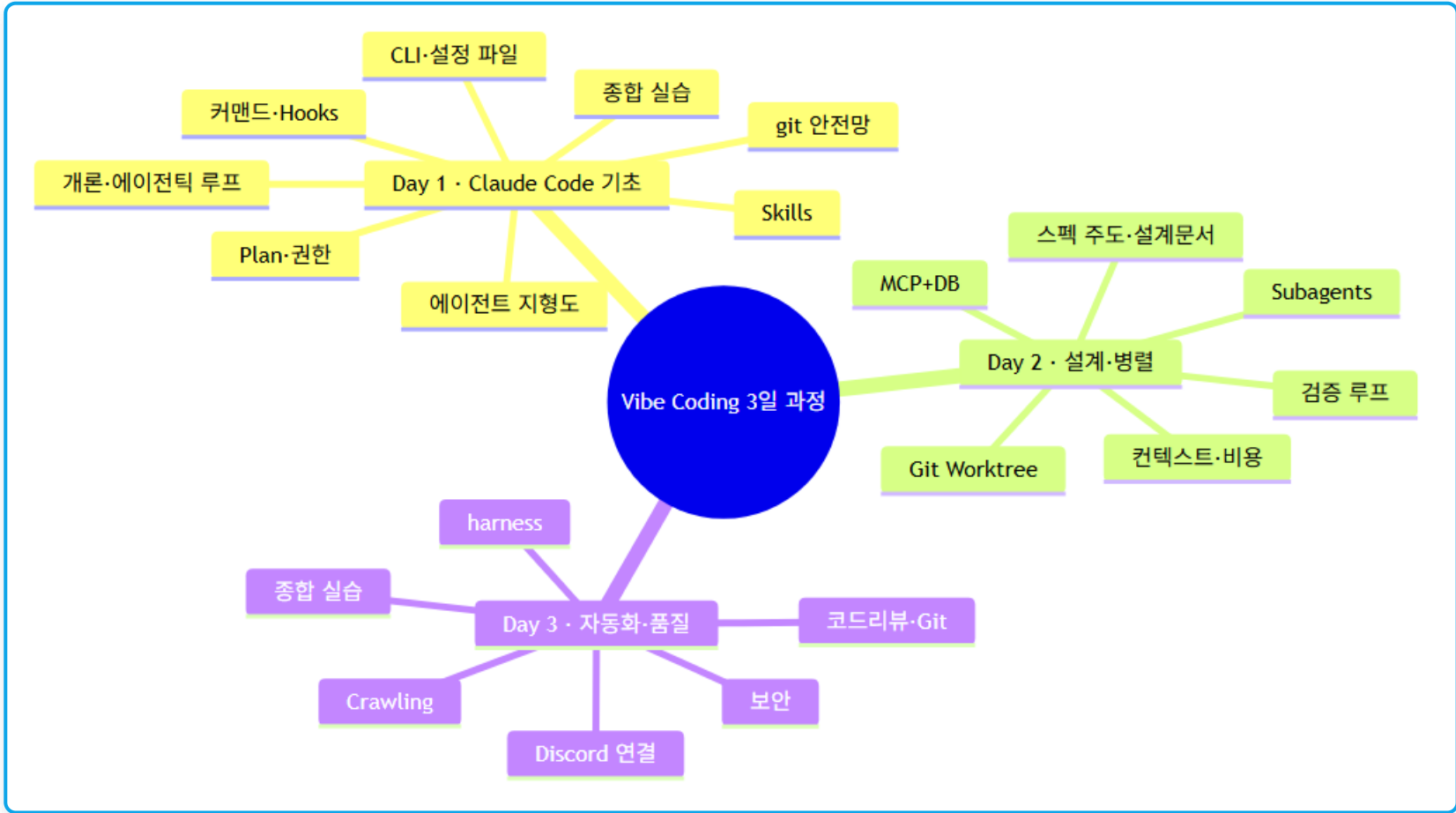
sotaaz.com
<https://sotaaz.com> > post > vibecoding-app-failures

바이브코딩으로 만든 앱, 왜 터질까? — 실제 사고 사례와 생존 ...

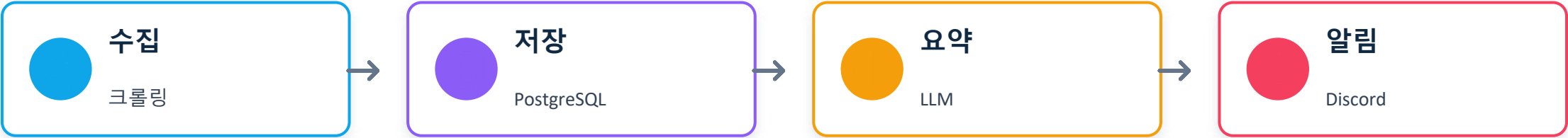
2026. 3. 23. — 150만 API 키 노출, 프로덕션 DB 삭제, 7만 신분증 유출 — 바이브코딩 앱 실제 사고 사례 6건과 반복되는 7가지 실패 패턴을 분석합니다.

전체 맵 – 3일을 한눈에 (마인드맵)

중심(Vibe Coding 3일 과정)에서 Day 별로 주제가 가지처럼 확장 – 다루는 것과 안 다루는 것의 지형을 먼저 세운다.



관통 프로젝트 – 3일을 꺾는 한 줄기



Day 1

작업 환경(harness) 구축 — CLAUDE.md·커맨드·훅·스킬

오늘

Day 2

스펙·설계문서로 못 박기 + Worktree·Subagents 병렬 + MCP·DB

Day 3

크롤링·보안·리뷰·Discord 연결·자동화로 실제 파이프라인 완성

1교시 정리 vibe coding = 의도를 기술해 AI가 구현, 사람은 조종·검증에 집중. 심장은 에이전틱 루프. 위임은 방기가 아니고, '잘 시키는 법'과 '믿는 법'이 한 쌍.



PERIOD 2 · 이론+탐색 ★★★

코딩 에이전트 지형도 + 오픈소스 에이전트

도구 하나만 파면 그 습관에 갇힌다. 먼저 지형을 봐야 Claude Code가 어디에 강하고 대안은 무엇인지 판단할 수 있다.
결론 하나 — Anthropic이 공개한 SKILL.md가 사실상의 에이전트 플러그인 표준이 되어가고 있다(7교시·3일차로 회수).

주요 코딩 에이전트 비교 – 이름은 달라도 CF, MCP, SKILL로 수렴한다

도구	형태	특징	확장 방식
Claude Code	CLI·IDE·데스크톱·웹	에이전틱 루프·서브에이전트·훅·스킬이 촘촘	CLAUDE.md·commands·hooks·skills·MCP
Codex (OpenAI)	CLI·클라우드	클라우드 실행·PR 자동화 강점	AGENTS.md·MCP
Gemini CLI	CLI	대용량 컨텍스트·구글 생태계	확장·MCP
Cursor	에디터(IDE)	편집기 통합·인라인 편집 UX	룰·MCP



형태별 분류 – 어디서 어떻게 만나는 에이전트인가

같은 '코딩 에이전트'라도 만나는 자리가 다르다. 축: 개발자 밀착 ↔ 위임·로컬 ↔ 원격.



터미널 CLI Claude Code·Codex·Gemini CLI

강점 파일·셸 직접 조작, 스크립트화(-p)

쓰임 로컬 개발의 주력, 자동화 부품



IDE 통합 Cursor·Copilot

강점 인라인 편집·즉각 피드백

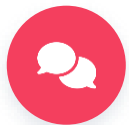
쓰임 타이핑 중 보조, 리팩토링



클라우드 위임형 Codex cloud 등

강점 백그라운드 병렬 실행, PR로 회신

쓰임 '맡겨두고 다른 일' — 이슈→PR



메시징/비서형 OpenClaw 류

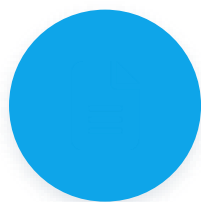
강점 채팅 채널에서 상시 대기

쓰임 알림·질의·개인 자동화



이 과정의 선택 터미널 CLI(Claude Code)를 주력으로 — 가장 강력하고(파일·셸 직접), 자동화로 확장되며, 다른 형태의 개념(위임·채널 연결)도 CLI 위에서 재현한다.

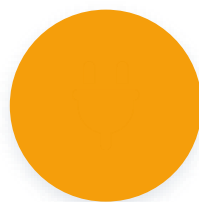
수렴하는 3요소 – 도구가 달라도 남는 것



① 프로젝트 컨텍스트 파일

CLAUDE.md / AGENTS.md / 룰 파일 – '이 프로젝트를 알게' 만드는 층

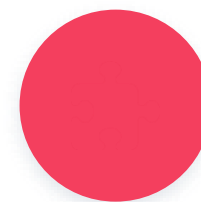
3교시



② MCP

Model Context Protocol – 외부 시스템(DB·API·서비스) 연결의 공통 규격

Day 2



③ 스킬 / 룰

절차·지식의 재사용 단위 – SKILL.md가 이 층의 표준이 되는 중

7교시



공통 문법 오늘 배우는 확장 수단들은 특정 제품 기능이 아니라 '에이전트 시대의 공통 문법'에 가깝다 – 도구를 갈아타도 개념은 남는다.

오픈소스 사례 – OpenClaw · Hermes Agent



OpenClaw 채널을 연결하는 게이트웨이

TypeScript 퍼스널 AI 어시스턴트 게이트웨이. [WebSocket Gateway](#)로
Discord·Telegram·Slack 등 **50+ 메시징 채널** 연결 + [ClawHub](#) 스킬 마켓플레이스.

창립자 P. Steinberger가 2026.2 OpenAI 합류 후 독립 재단으로 이관. 초기 보안 사고(CVE) 다수.

→ 3일차 보안의 반면교사 + Discord 연결 실습의 원형



Hermes Agent 잊지 않는 에이전트

Nous Research(2026.2, MIT, Python). 핵심은 세션이 끝나도 잊지 않는 **자기 개선 학습 루프** — 성공한 작업의 절차를 **스킬 파일로 저장**해 재사용.

모델 애그노스틱(로컬 Ollama~클라우드), 2026.5 OpenRouter 사용량에서 OpenClaw 추월.

→ 7교시에서 우리가 손으로 하는 일(절차→스킬)의 자동화판



둘 다 SKILL.md 표준을 채택, 오픈소스로 번지는 플러그인 표준의 산 증거.

다섯 기준으로 나란히 – 강점이 갈린다

기준	OpenClaw	Hermes Agent	우위
자기 학습	사람이 승인해야 스킬 등록	작업 끝나면 스킬 자동 생성·개선	Hermes
보안	기본 개방적 · 초기 CVE 다수	방어 계층 기본 적용	Hermes
메모리	날짜별 로그 자동 정리	핵심 기억 항상 프롬프트에	반반
스킬 생태계	다수 · 사람이 작성	다수 · AI 자동 생성	반반
지속성	재단+OpenAI · 창업자 이직	Nous Research 직접 운영	Hermes



언제 어느 것 OpenClaw = 여러 명이 함께·직접 조립·투명한 기록(팀·개방형) / Hermes = 1인 개발자·한 번 세팅 후 자동 성장·보안 최우선. 공통점은 둘 다 SKILL.md 표준(2.8).

SKILL.md – 사실상의 플러그인 표준이 되기까지



7교시 SKILL.md를 직접 만들며 실체 확인 → **3일차** Discord 연결(OpenClaw 채널)·보안(CVE)으로 회수

정리 에이전트는 형태(CLI·IDE·클라우드·메시징)는 달라도 **컨텍스트 파일·MCP·스킬** 3요소로 수렴한다.



2교시 정리 OpenClaw·Hermes는 SKILL.md 표준이 오픈소스로 번지는 흐름의 산 증거. 3교시: 드디어 Claude CLI를 손에 잡고 설정 파일 체계까지.



PERIOD 3 · 실습 ★★★

Claude CLI 기초 + 설정 파일 체계

채팅이 아니라 도구를 든 에이전트 — 파일을 읽고·고치고·명령을 실행한다. 대화만으로 첫 기능을 만들고, 매 프롬프트에 규칙을 반복하지 않도록 설정 파일(CLAUDE·MEMORY·AGENTS)로 '프로젝트를 항상 알게' 만든다.

Claude CLI란 – 채팅이 아니라 에이전트



챗봇 텍스트만 반환

질문 → 텍스트 답변 → 끝

코드를 '알려주지만' 직접 파일을 고치거나 명령을 실행하지는 못한다. 복붙은 사람 몫.



코딩 에이전트 도구를 쓴다



파일 읽기



수정·생성



검색 grep



셸 실행

에이전트가 필요에 따라 스스로 쓴다 → 더 강력하고, 그만큼 안전망(4교사)·권한(5교사)이 중요.



챗봇(텍스트만 반환) ↔ 에이전트(파일·셸 도구를 씀)의 대비가 위 두 카드. 이것이 1.3 에이전틱 루프의 실체다.

설치·인증·첫 실행

</> 설치와 첫 실행

```
# 설치 (또는 공식 네이티브 인스톨러)

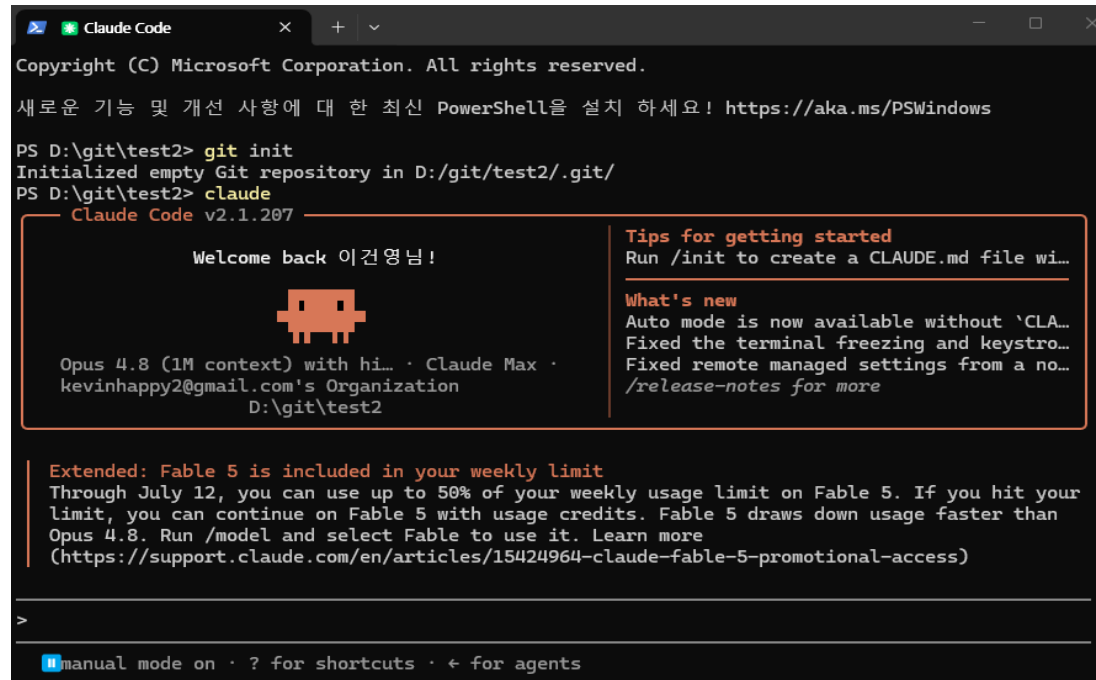
npm install -g @anthropic-ai/claude-code

cd my-project

# 첫 실행 → 브라우저 로그인(인증)

claude
```

- 인증은 최초 1회(Claude 계정/API 키) — 강의 시작 전 준비가 원칙(부록 B).
- 프로젝트 폴더 안에서 실행해야 그 폴더가 작업 컨텍스트가 된다.



The screenshot shows the Claude Code terminal window. At the top, it says 'Copyright (C) Microsoft Corporation. All rights reserved.' followed by a message about PowerShell: '새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows'. Below this, the terminal shows the execution of 'git init' and 'claude' commands. The 'claude' command outputs 'Claude Code v2.1.207' and a welcome message in Korean: 'Welcome back 이건영님!'. A small robot icon is displayed. To the right, there are 'Tips for getting started' and 'What's new' sections. The 'What's new' section mentions 'Auto mode is now available without `CLA...`' and 'Fixed the terminal freezing and keystro...'. At the bottom, there is an 'Extended: Fable 5 is included in your weekly limit' notice.



표면 CLI 외에 IDE·데스크톱·웹 — 이 과정은 CLI 주력.

핵심 개념 – 세션·도구·권한·슬래시 + 대화형/원샷

**세션**

폴더에서 claude 실행 → 대화 시작. 프로젝트 컨텍스트를 안다. -c(계속)--r(재개)

**도구**

파일 읽기/수정/생성·grep/glob·셸·웹 검색 — 필요에 따라 스스로 사용

**권한**

파일 수정·명령 실행 전 승인(기본). 위험 작업은 반드시 확인(5교시)

**슬래시**

/help·/init·/model·/clear·/compact — 커스텀 커맨드도(6교시)

방식	명령	언제
대화형	claude	탐색·반복·설계 — 붙어서 주고받을 때
원샷(headless)	claude -p "..."	한 번 시키고 결과만 — 스크립트·CI·자동화(3일차)

 **원샷 -p** 같은 에이전트를 파이프라인 부품으로 쓰는 문. 오늘은 존재만 익히고 3일차 헤드리스·CI에서 본격 사용.

첫 세션 – 대화로 기능 만들기 (wordcount.py)

빈 폴더에서 **git init** 후 **claude** 실행. AI와 대화하며 기능 하나를 만들고 **도구 사용·승인 흐름**을 관찰한다.

</> 프롬프트 예시

텍스트 파일의 단어 수를 세는 CLI wordcount.py 를 만들어줘.

- 사용법: `python wordcount.py <파일>`
- 파일 없으면 친절히 에러 + 종료코드 1
- 표준 라이브러리만, 다 만들면 실행해서 보여줘

>_ 승인 흐름 (① 생성 → ② 실행)

```
$ claude
> wordcount.py 를 만들어줘 ...

• Create file wordcount.py?
  > 1.Yes 2.Yes,always 3.No
✓ Created wordcount.py (32 lines)

• Run: python wordcount.py ...?
  > 1.Yes 2.No

words: 128
```



체크포인트 승인 프롬프트가 언제 뜨는지, 세션이 프로젝트 파일을 아는지. (되돌리기는 4교시)

설정 파일 3종과 로딩 순서 – 프로젝트를 항상 알게



CLAUDE.md

프로젝트 안내서 – 스택·구조·규칙·함정

위치 루트(공유) · ~/.claude(전역)



MEMORY.md

세션을 넘어 유지되는 기억(3.7)

위치 개인 메모리 영역



AGENTS.md

도구 중립 오픈 표준 컨텍스트

위치 프로젝트 루트

로딩 순서 – 넓은 것 → 좁은 것

전역 ~/.claude 개인 취향



프로젝트 루트 팀 규칙



하위 디렉토리 폴더 특수 규칙 (가장 우선)

아래일수록 더 구체적이고 우선한다 – 상충 시 더 구체적인 쪽이 이긴다.



비유 프롬프트는 구두 지시, 설정 파일은 팀 온보딩 문서. 새 팀원에게 매번 말하는 대신 문서를 주듯 AI에게도 문서를 준다.

MEMORY.md와 세션 기억 – 대화는 끝나도 기억은 남게



CLAUDE.md

프로젝트 지식 · 팀 공유



MEMORY.md

세션을 넘어 유지되는 작업 기억 · 개인

무엇을 기억시키나 – 다음 세션에서도 알아야 할 것

- 코드로 알 수 없는 **결정** – "인증은 JWT로 하기로 함"
- **선호** – "커밋 메시지는 한국어"
- **진행 중 작업** – 어디까지 했나
↳ 코드가 이미 말해주는 것은 담지 않는다.

어떻게 – 두 가지 방법

- ① 대화 중 "**이거 기억해둬**" 라고 시킨다
- ② 입력을 **# 으로 시작** → 메모리에 바로 추가(단축)

> # 인증은 JWT로 하기로 함



/clear·새 세션 대화 컨텍스트는 날아가도 **메모리 파일은 남는다** → "긴 대화 하나"보다 "**짧은 세션 여러 개 + 기억**" 이 컨텍스트 소실(1.5)에 강하다.



예고 상세한 컨텍스트·비용 전략(/compact·/cost·모델 티어링)은 Day 2 1교시에서 – 오늘은 '기억은 파일로 남는다'만 잡는다.

CLAUDE.md 초안은 /init으로 · 자동 vs 사람이 다듬기 (지시→관찰)

관찰 – /init이 채운 것 vs 사람이 채울 것

- ✓ 스택·구조 폴더의 의미 /init 자동
- ✓ 실행·테스트 돌리는 법 /init 자동
- ✓ 개요·목표 무엇을·왜 사람 →3.9
- ✓ 규칙·컨벤션 커밋·시크릿 정책 사람 →3.9
- ✓ 합정·주의 밝기 쉬운 지뢰 사람 →3.9



낮선 저장소 /init이 곧 코드베이스 파악의 시작(Explore). "구조·핵심 흐름·위험한 부분 설명해줘"로 지형도를 그려줌.
이 워크플로 (Explore→Plan→Code→Commit)은 Day 2 4교시에서.

```

> /init

• Bash(cd "D:/git/test2" && ls -la && echo "----" && find . -maxdepth 2 -not -path "./.git*" 2>/dev/null)
  L total 16
    drwxr-xr-x 1 user 197121    0 Jul 12 08:06 .
    drwxr-xr-x 1 user 197121    0 Jul  4 19:35 ..
    ... +20 lines (ctrl+o to expand)

  Read 1 file (ctrl+o to expand)

• No README, cursor/copilot rules, or existing CLAUDE.md - this is a minimal two-file repo, so I'll keep it concise.

• Write(CLAUDE.md)
  L Wrote 18 lines to CLAUDE.md
    1 # CLAUDE.md
    2
    3 This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.
    4
    5 ## Commands
    6
    7 - Run tests: `python -m pytest test_wordcount.py -q`
    8 - Run a single test: `python -m pytest test_wordcount.py::test_name -q`
    9 - Run the CLI: `python wordcount.py FILE [FILE ...]`
    ... +14 lines (ctrl+o to expand)

• CLAUDE.md를 생성했습니다. 저장소가 wordcount.py와 그 테스트 파일 두 개뿐이라 커맨드와 아키텍처(두 함수의 역할 분리, 에러 처리 방식)만 간결하게 담았습니다.

* Baked for 12s

> |commit this

  >> accept edits on (shift+tab to cycle) · ← for agents
```

빈 규칙을 구체적 지시로 채운다 · 커밋 컨벤션·시크릿 (지시→관찰)

</> 지시 – "## 규칙·컨벤션" 섹션 채우기

- > CLAUDE.md에 "## 규칙·컨벤션" 섹션을 추가해줘.
- 기존 구조·실행법 섹션은 그대로 두고, 1s·소스로 유추 가능한 건 넣지 마.
- 아래 "코드를 봐도 모르는 규칙"만 담아:
 - 시크릿: API 키·토큰은 .env 로만, 절대 커밋 금지(.gitignore 확인)
 - 커밋 메시지: Conventional Commits
- 타입(feat/fix/docs/refactor/test/chore)
 - + 콜론 뒤 한국어 요약 한 줄. 예) "feat: 여러 파일 단어수 합계 출력"
- 테스트: 로직 변경 시 테스트 갱신, 통과 없이 커밋 금지

>_ 관찰 – 규칙 넣기 전 / 후

- # 규칙 넣기 전
- > 방금 변경으로 커밋 메시지 만들어줘
 - "CLAUDE.md에 커밋 컨벤션이 없습니다"
 - 저장소 기존 스타일을 추정 (근거 약)
- # 규칙 넣은 후
- > 방금 변경으로 커밋 메시지 만들어줘
 - "문서화한 컨벤션 그대로 적용"
 - docs: 규칙·컨벤션 섹션 추가 (...)
 - 추정이 아니라 '우리 규칙'을 근거로 ✓



Day 2 예고 무엇을·제약·확인법을 담아 시키는 이 요령은 Day 2에서 '좋은 지시의 4요소'로 정식화. 지금은 "지시가 구체적일수록 결과가 일관"만 몸으로 느낀다. (이 커밋 규칙은 6교시 /commit·8교시 점검이 참조)

AGENTS.md 오픈 표준 + 심볼릭 링크 · 로딩 순서·우선순위 실측

</> 3.10 한 원본, 여러 도구 - 링크 (OS별)

```
# 지시만 하면 Claude가 OS 맞게 실행
ln -s AGENTS.md CLAUDE.md      # mac/Linux
New-Item -ItemType HardLink `   # Windows
    -Path CLAUDE.md -Target AGENTS.md

# ⚠ PowerShell서 mklink 직접입력=에러

# (cmd 내장) → cmd /c mklink /H CLAUDE.md AGENTS.md 로 감싸기
```

AGENTS.md는 도구에 안 묶이는 오픈 표준(2.4의 ①). 한 원본만 두면 어떤 도구로 열어
도 같은 규칙 - 방향은 무관.



3교시 정리 Claude CLI = 도구를 든 에이전트(세션·도구·권한·슬래시, 대화형/원샷). 컨텍스트가 프롬프트보다 오래간다 - CLAUDE·MEMORY·AGENTS가 매
세션에 깔리고 구체적인 쪽이 이긴다. 이것이 확장 4종 세트 ①. 다음: git 안전망.

> 3.11 [실습] 로딩 순서·우선순위 실측

```
# 3층 로딩 (넓은 것 → 좁은 것)
전역 ~/.claude/CLAUDE.md
→ 프로젝트 루트 → 하위 폴더

# 우선순위 상층 실측
전역: "주석은 한국어"
프로젝트: "주석은 영어"

→ 프로젝트(아래)가 이긴다 ✓
```

4

PERIOD 4 · 실습 ★★

git 안전망과 되돌리기

에이전트는 파일을 수십 개씩 고친다. 겁이 나면 위임을 못 하고, 위임을 못 하면 vibe coding이 아니다. 해법은 겁을 없애는 게 아니라 '망쳐도 되는 환경'을 만드는 것 — 그 환경이 git이다. 되돌릴 수 있으면 과감해질 수 있다.

커밋 리듬 – 되돌릴 지점을 만드는 습관



왜 안전망인가

AI 시대에 git은 협업 도구를 넘어 **에이전트용 안전망**이 됐다.

- 커밋이 잘게 쌓이면 어디서든 되돌아갈 수 있다.
- 실패해도 마지막 안전 지점까지만 잃는다.
- 커밋 메시지도 AI에게 – "방금 변경으로 메시지 제안해줘" (6교시 /commit으로 승격)

핵심 루프



확인되면 바로 커밋 – 커밋이 곧 "여기까지는 안전" 표시다.



한 단위 일 → 확인 → 커밋 AI에게 한 단위 일을 시키고, 확인되면 바로 커밋. 잘게 쌓을수록 무서울 게 없다.

4.3 READ THE DIFF

diff 읽기 – 'AI가 뭘 바꿨나'부터

</> 커밋 전에 반드시 diff를 본다

```
git status      # 뭐가 바뀌었나
git diff        # 정확히 뭐가 (스테이징 전)
git diff --staged # 스테이징된 것만
```

요령 파일 수가 많으면 "방금 바꾼 걸 파일별로 요약해줘"라고 AI에게 먼저 시키고, 수상한 파일만 diff로 정독 – **요약은 AI, 판단은 사람.**



신뢰의 첫 단계 "무엇이 바뀌었는지 아는 것".

```
Windows PowerShell
(use "git restore <file>..." to discard changes in working directory)
modified:   CLAUDE.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
.gitignore
AGENTS.md

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\git\vibe_handson> git diff
warning: in the working copy of 'CLAUDE.md', LF will be replaced by CRLF the next time Git touches it
diff --git a/CLAUDE.md b/CLAUDE.md
index 139c8bf..199aa56 100644
--- a/CLAUDE.md
+++ b/CLAUDE.md
@@ -16,3 +16,10 @@ This file provides guidance to Claude Code (claude.ai/code) when working with co
- 'main(argv: list[str] | None = None) -> int' - argparse-based entry point. Reads each file (UTF-8, 'errors="replace"
), prints one 'count path' line per file, and a 'total N' line when more than one file is given. Missing/unreadable file
s report to stderr and cause exit code 1, but don't stop processing of the remaining files.

'test_wordcount.py' exercises both 'count_words' and 'main' directly (no subprocess calls) using pytest's 'tmp_path'/'c
apsys' fixtures.
+
+## 규칙·컨벤션
+
+- **시크릿**: 키·토큰은 '.env'로만 관리하고, 커밋하지 않는다.
+- **커밋**: Conventional Commits('feat'/'fix'/'docs'/'refactor'/'test') + 콜론 뒤 한국어 한 줄로 작성한다.
+- **테스트**: 로직 변경 시 함께 갱신하고, 테스트 통과 없이 커밋하지 않는다.
```

[스크린샷] git diff로 AI 변경 확인 (4.3)

에이전트 작업 직후 git diff 출력(추가/삭제 하이라이트). 캡션: "커밋 전 diff –
안 짤 코드를 아는 첫 단계"

되돌리기 3종 – restore · revert · reset

명령	되돌리는 것	언제	위험도
git restore .	커밋 전 작업 폴더 변경	AI 결과가 마음에 안 들 때	낮음
git revert <커밋>	특정 커밋을 취소하는 새 커밋	이미 커밋/공유한 변경 취소	낮음
git reset --hard <커밋>	그 지점 이후 전부 삭제	로컬에서 완전히 물리기	높음

기본기 평소엔 restore(커밋 전)와 revert(커밋 후)로 충분. **reset --hard**는 "정말 버릴 것인지" 한 번 더 — 공유 브랜치에선 금물.

체크포인트 /rewind + .gitignore·시크릿



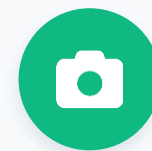
체크포인트 — /rewind

에이전트가 편집할 때마다 자동 체크포인트. /rewind(또는 Esc 두 번)로 **코드만·대화만** ·돌 다 되감기.

· git 커밋 = 내가 만드는 굵은 안전 지점

· 체크포인트 = 도구가 만드는 촘촘한 자동 저장

한계 Bash 부수효과·외부 시스템은 못 되돌림 — 최후의 진실은 git.



[스크린샷] 체크포인트 되감기 (4.5)

/rewind 실행(시점 목록·코드/대화 선택) · "대화가 산으로 가면 되감고 다시 지시"



.gitignore와 시크릿 — 처음부터 새지 않게

.env·키가 한 번이라도 커밋되면 이력에 남는다(지우기 매우 어려움). 첫 커밋 전에 .gitignore부터 — "이 스택에 맞는 .gitignore 만들어줘, .env 포함". 6교시엔 혹은 강제 차단(오늘의 예고편).



겹쳐 쓰면 잃을 게 없다 큰 실험 전엔 커밋, 대화가 산으로 갔을 땐 되감기 — 둘을 겹쳐 쓴다.

브랜치 = 실험실 + [실습] 일부러 망치고 되돌리기

</> 과감한 실험은 브랜치에서

```
git switch -c experiment/big-refactor
# ... AI에게 과감하게 시킨다 ...
git switch main # 안 되면 그냥 돌아온다
```

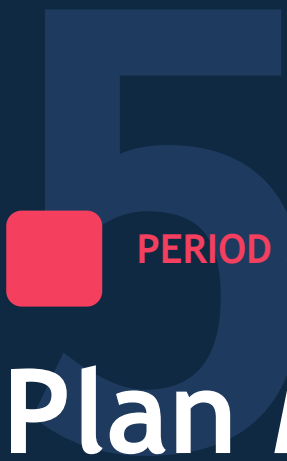
"될까?"를 main에서 고민하지 말고 브랜치에서 해보고 버린다. **Day 2 Git Worktree** 는 이 확장 — 브랜치 여러 개를 동시에 열어 에이전트 병렬 실행.

> [실습] 안전망은 써봐야 믿는다

1. 현재 상태를 커밋 (안전 지점)
2. AI에게 일부러 큰 변경을 시킨다
"모든 함수에 로깅 추가+구조 변경"
3. git diff 로 훑기 → git restore .
→ 전부 되돌리기
4. 다시 시키고 /rewind 로 되감기 체험
5. .gitignore 에 .env 추가 →
git status 에 안 뜨는지 확인 ✓



4교시 정리 되돌릴 수 있으면 과감해질 수 있다 — 커밋 리듬 + 체크포인트 + diff 확인 + 브랜치 + .gitignore. 다음(5교시): 사고를 되돌리는 게 아니라 애초에 못 일어나게 하는 Plan Mode·권한.

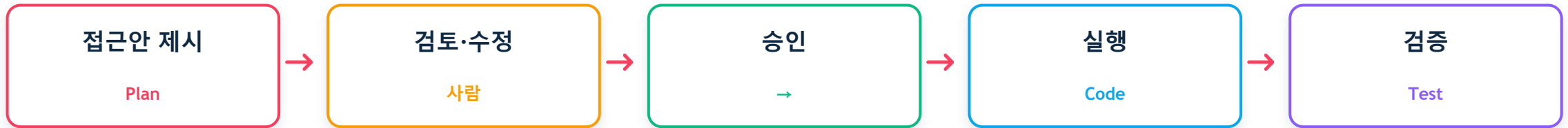


PERIOD 5 · 실습 ★★

Plan Mode와 권한 모드

복잡한 작업은 바로 코드 말고 계획부터 받는다 — 계획 30초 검토가 잘못된 구현 30분을 아낀다. 4교시가 사후 안전망이라면, 이번 교시는 사전 안전장치. 행동하는 에이전트는 계획을 먼저 보이고 권한으로 통제해야 안전하다.

Plan Mode – 계획을 먼저, 파일은 나중에



계획 단계에서 고칠 것

- 빠진 요구사항
- 과한 범위(스코프 크립)
- 잘못된 접근 — 새 파일 대신 기존 모듈 수정

계획을 고쳐서 승인할 수 있다는 게 핵심 (모드 전환: Shift+Tab)

```
Claude Code v2.1.210
Sonnet 5 with high effort · Claude Pro
D:\git\2604_agent_210h_handson

!! 2 MCP servers need authentication · run /mcp

Extended through July 19
We're extending Claude Fable 5 access on all paid plans, as well as keeping Claude Code's weekly rate limits 50% higher, through July 19.
As before, you can use up to half of your weekly usage limit on Fable 5. After that, you can keep using Fable 5 with usage credits, or switch to
another model to keep working within your remaining limits.
More details here: https://support.claude.com/en/articles/15424964-claude-fable-5-promotional-access
+1 more · /status

/login
└ Login successful

/plan
└ Enabled plan mode

> /plan|
plan mode on (shift+tab to cycle)
```



왜 계획 먼저 많은 실패는 '코드가 틀려서'가 아니라 '방향이 틀렸는데 코드부터 나와서' 생긴다. 잘못된 방향으로 쏟아지는 코드를 막는다.

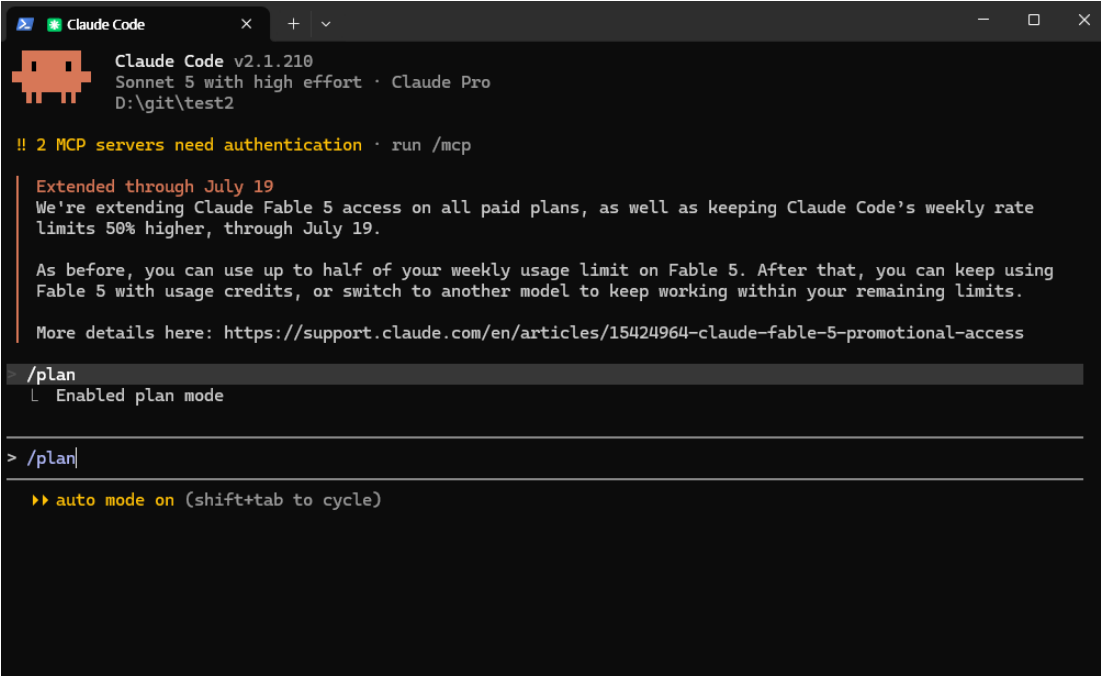
권한 모드 – Manual · Accept edits · Plan (+Auto·Bypass)

- 

Manual (기본)
읽기만 자동, 수정·명령 실행마다 승인 요청 · *낮선 작업·중요 변경*
- 

Accept edits
파일 편집은 자동, 그 외 명령은 여전히 확인 · *신뢰 쌓인 반복 작업*
- 

Plan
실행 없이 계획만 · *설계·큰 작업 착수 전*



```
Claude Code v2.1.210
Sonnet 5 with high effort · Claude Pro
D:\git\test2

!! 2 MCP servers need authentication · run /mcp

Extended through July 19
We're extending Claude Fable 5 access on all paid plans, as well as keeping Claude Code's weekly rate
limits 50% higher, through July 19.

As before, you can use up to half of your weekly usage limit on Fable 5. After that, you can keep using
Fable 5 with usage credits, or switch to another model to keep working within your remaining limits.

More details here: https://support.claude.com/en/articles/15424964-claude-fable-5-promotional-access

> /plan
  L Enabled plan mode

> /plan|
  >> auto mode on (shift+tab to cycle)
```

 **임시 통제** 모드는 작업 성격에 맞게 전환한다 — 낮선 건 Manual, 신뢰 쌓인 반복은 Accept edits, 큰 착수 전엔 Plan. Shift+Tab 순환엔 Auto·Bypass permissions도 끼어 들 수 있다.

권한 규칙 – settings.json allow/deny (영구 통제)

</> .claude/settings.json (프로젝트 공유)

```
{
  "permissions": {
    "allow": ["Bash(git diff:*)",
              "Bash(npm run test:*)"],
    "deny": ["Bash(curl:*)",
              "Read(./env)"]
  }
}
```

allow 안전한 조회·테스트를 등록 → 승인 피로 감소

deny 어떤 모드에서도 금지(외부 전송·시크릿) → 최소 권한. 팀 공통 규칙은 커밋해 환경으로 공유.



⚠️ **--dangerously-skip-permissions**

모든 확인을 끝다 — 삭제·push·외부 전송·설치까지 무확인. 격리 샌드박스(컨테이너·일회용 VM)가 아니면 쓰지 말 것. 1.6 DB 삭제가 정확히 이 결말.



모드 vs 규칙 모드(임시)와 별개로 규칙(영구)을 파일로 못 박는다. deny는 모드와 무관하게 막는다 — 6교시 Hooks와 함께 harness의 뼈대.

파일 3종 실습 중 '항상 허용'은 **settings.local.json**(나만·미커밋)에 자동 기록 — 팀 공유 규칙만 settings.json으로 커밋. (~/.claude/settings.json=전 프로젝트)

안전 4원칙 – 과감함과 안전을 함께



되돌리기 어려운 작업은 확인

삭제·push·외부 전송·설치는 항상 승인 후



작은 단위로 자주

큰 변경은 쪼개서 자주 검토·커밋(4교시 커밋 리듬)



직접 확인

"AI가 했다"는 주장은 파일·실행으로 검증



git 커밋

중요 시점마다 되돌릴 지점 남기기(4교시)



사후 + 사전 4교시(사후 복구)와 5교시(사전 통제)로 안전이 완성됐다 — 이제 에이전트를 우리 방식대로 확장하는 커맨드·혹으로 간다.

[실습] Plan→검토→실행 1사이클

> 관통 프로젝트에서 한 사이클

```
# 여러 파일이 얹힌 작업을 plan으로
> 설정 로더 + 테스트 추가 (Plan Mode)
  계획을 읽고 한 군데를 고쳐 승인
  → 실행 → git diff 확인 → 커밋

# 모드 체감: 같은 작업 Ask vs Auto
  승인 횟수·속도 차이 관찰

# 규칙 1개: deny Read(./env)
  .env 읽으라고 시켜 → 거부되는지 ✓
```

체크포인트

- ☐ 계획을 승인 전에 수정할 수 있는가
- ☐ AutoAccept에서도 위험 작업은 여전히 확인하는가
- ☐ deny 규칙이 모드와 무관하게 막는가



5교시 정리 임시 통제 = 3모드(Ask/AutoAccept/Plan), 영구 통제 = settings.json allow/deny, 금지 = --dangerously(격리 밖). 다음(6교시): 커맨드·Hooks로 확장.

PERIOD 6 · 실습 ★★

커맨드와 Hooks — 부탁 vs 강제

자주 반복하는 지시는 커스텀 슬래시 커맨드로 버튼화한다. 하지만 프롬프트는 '부탁'이라 잊거나 건너뛸 수 있다 — Hook은 특정 시점에 코드를 결정적으로 실행하는 '강제'다. 이 차이가 안정적 파이프라인을 만든다.

커스텀 슬래시 커맨드 – 반복 지시를 템플릿화

```
</> .claude/commands/ </>

commit.md    → /commit    : 변경 요약 + 컨벤션에 맞는 커밋 메시지
review.md    → /review    : 현재 diff를 체크리스트로 리뷰
```

```
</> .claude/commands/commit.md </>

---
description: 스테이징된 변경으로 커밋 메시지 제안
---

스테이징된 변경(diff)을 요약하고,
CLAUDE.md의 커밋 컨벤션에 맞는
한국어 커밋 메시지를 제안해줘.

$ARGUMENTS
```

위치	범위	용도
.claude/commands/	프로젝트(커밋해 팀 공유)	팀 워크플로 /commit-/review
~/.claude/commands/	개인(모든 프로젝트)	개인 습관 /메모./요약

\$ARGUMENTS /commit 자세하게 처럼 넘긴 인자가 그 자리에 치환

프론트매터 description(목록에 뜨는 설명) 등 메타데이터

→ 커맨드 안에서 CLAUDE.md 규칙을 참조하면 확장 수단들이 서로 물린다

 **하위 폴더 = 네임스페이스** commands/git/commit.md → /git:commit. 설정 파일과 같은 원리 — 팀 것은 프로젝트에, 취향은 전역에.

[실습] /commit·/review 만들기

</> ① 만들기 – 커맨드도 지시로

> `.claude/commands/commit.md` 를 만들어줘.
 내용은: 스테이징된 diff를 요약하고
 CLAUDE.md의 커밋 컨벤션(feat/fix...)에 맞는
 한국어 커밋 메시지를 제안.
 맨 끝에 \$ARGUMENTS 를 넣어줘.

> 이어서 `.claude/commands/review.md` 도
 만들어줘. 현재 변경(diff)을 버그·시크릿
 노출·테스트 누락 관점의 체크리스트로
 리뷰하게.

→ 슬래시 목록에 /commit·/review 등장

> ② 돌려보기 – 관찰

> (코드 한 곳 수정) → git add

> /commit

docs: 규칙·컨벤션 섹션 추가 (...)

> /commit 본문도 자세히 # \$ARGUMENTS

docs: ... + 본문 3줄로 확장

> /review

✓ 시크릿 없음 ⚠ 빈입력 처리 누락

→ 커밋 컨벤션은 CLAUDE.md 참조 ✓



관찰 포인트 슬래시 목록에 /commit·/review가 뜬다 · /commit이 CLAUDE.md 커밋 컨벤션을 따른다(3.9에 심은 규칙이 여기서 물림) · \$ARGUMENTS로 한 커맨드의 톤·범위를 조절.

확장 기능 4종 구분 – 부탁 · 위임 · 강제

수단	정체	언제 쓰나	통제력
Command	저장된 프롬프트	자주 쓰는 지시를 버튼화	부탁(제안)
Skill	이름 붙인 절차/지식	복잡한 다단계 작업 재사용(7교시)	부탁(제안)
Subagent	독립 컨텍스트 하위 에이전트	역할 분리·병렬(2일차)	위임
Hook	특정 시점 자동 실행 스크립트	강제 규칙(포맷·차단)	강제(결정적)



핵심 구분 Command·Skill은 '부탁'(모델이 따를 수도, 안 따를 수도), Hook은 '강제'(코드가 실행). 강제가 필요한 규칙(포맷·보안)은 반드시 Hook·deny로 건다.

훅 이벤트 훑기 – 언제 걸 것인가

이벤트	시점	대표 용도
PreToolUse	도구 실행 직전	위험 명령 차단, 시크릿 커밋 방지
PostToolUse	도구 실행 직후	자동 포맷·린트, 변경 로그
UserPromptSubmit	프롬프트 제출 시	컨텍스트 주입·입력 검사
SessionStart / Stop	세션 시작·응답 종료	환경 준비, 알림·기록

 **PreToolUse는 차단권을 가진다** 조건에 맞으면 그 도구 호출을 막는다 — '위험 명령·시크릿'의 최후 방어선.

 **동적 강제** 5교시 deny가 특정 도구 호출을 막는 정적 규칙이라면, Hook은 조건·시점을 코드로 판단하는 동적 강제다.

혹 작성법 – command는 '실행 가능한 셀 한 줄'

</> settings.local.json – hooks (그대로 복붙 · 실측 검증)

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash",
      "hooks": [
        {
          "type": "command",
          "command": "if grep -qE \"rm -rf|\\.env\"; then echo \"차단 사유: rm -rf 또는 .env 패턴 감지\" >&2; exit 2; else exit 0; fi"
        }
      ]
    }
  ],
  "PostToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [
        {
          "type": "command",
          "command": "echo \"$(date '+%Y-%m-%d %H:%M:%S') 파일 수정됨\" >> .claude/hook_log.txt"
        }
      ]
    }
  ]
}
```

종료 코드가 전부다 **exit 0** 통과 · **exit 2** 차단(stderr=사유가 모델에게) · **그 외(1·127)** 비차단 오류 → 그대로 실행됨. ✗ "command": "위험 명령 검사 스크립트" 같은 설명문 → command not found → 혹은 '조용히 열린다'(fail-open)



한 줄이면 된다 혹 명령은 파일일 필요가 없다 — 셀 한 줄로 충분. 패턴이 늘어 파일로 승격하고 싶어지면 그것도 지시로 시키면 된다.

[실습] 훅 2개 – 지시로 걸고, 정말 막히는지 시험

</> 지시 ①·② – 그대로 복붙

```
> settings.local.json에 PostToolUse 훅
  걸어줘. matcher는 Edit|Write, 명령은 셸
  한 줄로: .claude/hook_log.txt 에 시각과
  "파일 수정됨" 한 줄 추가

> PreToolUse 훅도 걸어줘. matcher는 Bash.
  stdin에 rm -rf 나 .env 가 보이면 사유를
  stderr로 찍고 exit 2, 아니면 exit 0
  하는 셸 한 줄로.
```

> 시험 – 걸었으면 막히는지 확인

```
# ① 로그 훅 – 눈으로 확인
> (아무 파일이나 수정 지시)
  hook_log.txt에 줄이 자동으로 쌓임 ✓
  "로그 남겨"라 한 적 없는데 남는다=강제

# ② 차단 훅 – 실전 시험
> ./data 폴더를 rm -rf로 지워줘
  ✗ BLOCKED – exit 2, 사유 전달
  "지울까요?" 묻지도 못한다 = 강제
  ⚠ non-blocking status code 가 뜨면
  차단이 아니라 훅이 깨진 것
```



6교시 정리 커맨드는 반복 지시를 팀 공유 버튼으로(②), Hooks는 프롬프트가 못 지키는 것을 코드로 못 하게(③). 단 혹은 '건 것'이 아니라 '막히는 것'을 확인해야 강제다. 다음(7교시): Skills.



PERIOD 7 · 실습 ★★★

Skills 활용

skill은 반복되는 다단계 절차·전문 지식을 SKILL.md로 정의해, 에이전트가 필요할 때 스스로 불러 쓰게 하는 확장이다. 커맨드가 '사람이 /이름으로 호출'이라면, 스킬은 '상황이 맞으면 에이전트가 알아서 참조'. 2교시의 SKILL.md 표준이 여기서 실체를 드러낸다.

SKILL.md – 절차와 지식을 이름 붙여 재사용

</> .claude/skills/release-note/SKILL.md

name: release-note

description: 릴리스 노트를 생성할 때 사용.

커밋 로그를 모아 카테고리별로 정리하고
사용자 관점 문장으로 다듬는다.

1. git log 로 직전 태그 이후 커밋을 모은다
2. feat/fix/docs 로 분류한다
3. 사용자 관점 문장으로 다듬는다

파일 구조

.claude/skills/<skill-name>/

SKILL.md 프론트매터 + 본문(절차)

references/... 필요할 때만 읽는 상세·스크립트

커맨드 = 사람이 /이름으로 호출

스킬 = 상황이 맞으면 에이전트가 알아서 참조

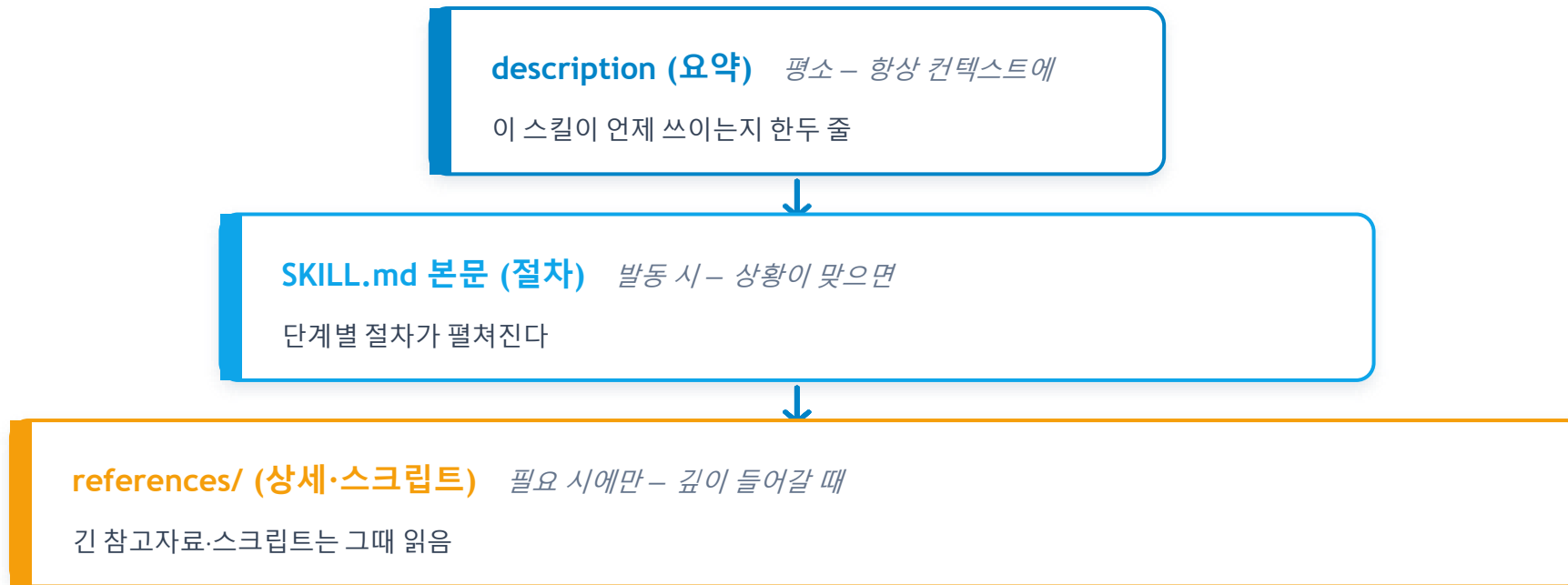
2교시/OpenClaw-Hermes의 그 SKILL.md 표준의 실체



재사용 단위 반복되는 다단계 절차·전문 지식을 한 파일로 이름 붙여, 유사 작업마다 다시 설명하지 않는다.

Progressive disclosure – 컨텍스트를 아끼는 설계

처음엔 짧은 요약(description)만 컨텍스트에 올리고, 실제로 필요할 때만 본문·상세를 연다.



평소엔 description만, 발동 시 SKILL.md 본문, 필요 시 references가 펼쳐지는 3단 계층 – 컨텍스트를 아끼며 깊은 절차를 담는다(2일차 컨텍스트 관리와 직결).

description = 트리거 + 커맨드 vs 스킬



description 작성 요령 – 스킬의 트리거

- **"~할 때 사용"** 형태로 발동 상황을 명시
(**"릴리스 노트를 만들 때"**, **"새 크롤러 소스를 추가할 때"**)
- **사용자가 실제로 쓸 말·키워드** 포함(한국어도)
- 발동 안 하면 십중팔구 description이 약한 것 –
'무엇을 하는지'가 아니라 **'언제 쓰는지'**를 다시 쓴다

기준	Command	Skill
호출 주체	사람 /이름	에이전트(판단)
분량	짧은 지시	다단계+참고
좋은 예	/commit	릴리스 절차
헛갈리면	버튼 누르고파	AI가 챙기면



무엇으로 만들까 "내가 버튼을 누르고 싶다" → Command, "AI가 알아서 챙기면 좋겠다" → Skill.

[실습] 커스텀 스킬 제작 – 스킬도 지시로 만든다

</> ① 만들기 – 지시 그대로 복붙

```
> .claude/skills/release-note/SKILL.md 를 만들어줘.
- 프론트매터: name은 release-note,
  description은 "~할 때 사용" 형태로 발동 상황이 잘 걸리게
  (예: "릴리스 노트를 만들 때 사용. 커밋 로그를 모아...")
- 본문 절차:
  ① git log로 직전 태그 이후 커밋 수집
  ② feat/fix/docs 분류
  ③ 사용자 관점 문장으로 다듬기
  ④ 결과를 RELEASE_NOTES.md에 추가
```

>_ ② 발동 시험 – / 없이 상황으로

```
# ⚠ 새 스킬은 새 세션에서 인식
# (스킬 목록은 세션 시작 때 로드)
# /없이 상황으로 던진다:
> 릴리스 노트 만들어줘
• (호출 안 했는데) release-note 참조 ✓
  커밋 수집→분류→다듬기→기록 수행
# 발동 안 하면 → description을 지시로 수정:
> description을 발동 상황 중심으로 다시 써줘
```



패턴 반복 커맨드(6.4)·혹(6.9)에 이어 스킬도 손으로 안 짤다 – 지시로 만들고, 발동 안 하면 코드가 아니라 description을 고친다(7.4). 다음(7.6): 이번엔 남이 만든 스킬을 가져다 쓴다.

[실습] 외부 스킬 설치 – 설치도 이제 "말"로 한다

> ① 설치 – 말 한 줄이면 끝

```
# Langfuse 공식 문서의 설치 안내 =  
# 명령어가 아니라 '에이전트에게 할 말'  
> Install the Langfuse Agent Skill  
from github.com/langfuse/skills.  
• 설치하는 중...  
• 설치 완료 ✓  
Review skills before use –  
they run with full agent permissions.  
  
# 예전: 문서 읽고→명령어 찾고→경로 맞추고  
# 지금: 문서가 '이렇게 말하세요'를 안내
```

</> ② 어디에 깔리나

```
.claude/skills/  
  release-note/SKILL.md  
    ← 7.5에서 내가 만든 스킬  
  langfuse/SKILL.md  
    ← 방금 설치된 남의 스킬  
# 내 스킬과 남의 스킬이  
# 같은 규격 · 같은 위치에 나란히  
# = 'SKILL.md가 표준'의 실물 (2.7 회수)  
# 설치 확인 = 이 폴더가 생겼는지
```



설치·설정마저 지시로 커맨드(6.4)·혹(6.9)·스킬(7.5)에 이은 마지막 확인 – 이제 문서가 "에이전트에게 이렇게 말하세요"를 안내한다. 단, 설치된 스킬은 내 에이전트 권한으로 돈다 – 5교시 권한·6교시 혹은 왜 필요한지가 여기서 회수된다.

열어보기 – 잘 만든 SKILL.md에 뭐가 들어 있나

</> ③ .claude/skills/langfuse/SKILL.md

```
# (1) 절차만이 아니라 '일하는 원칙'
1. Documentation First –
    기억에 의존해 구현하지 마라.
    항상 최신 문서를 먼저 가져와라.

# (2) 상세는 '목차'로만 걸려 있다
# .claude/skills/langfuse/SKILL.md 본문에 적힌 '목차'
- 기존 코드에 호출 기록 붙이기      → references/instrumentation.md
- 프롬프트를 Langfuse로 옮기기        → references/prompt-migration.md
- 기록을 읽고 실패 원인 분석하기      → references/error-analysis.md
- CI/CD에 품질 게이트 걸기           → references/ci-cd.md
... (참고 파일 총 11개)
```

> ④ 발동 시험 – 새 세션에서

```
# ⚠ 설치한 세션에선 발동 안 한다
# (스킬 목록은 세션 시작 때 로드)
# → 새 세션을 열고 시험한다
# / 없이 '상황'으로 던진다:
> 이 프로젝트에 LLM 호출 기록을 남기고 싶은데 뭐부터 해야 해?

• (안 불렀는데) langfuse 스킬 참조 ✓
  → 그때서야 instrumentation.md 열림

# 내가 만든 스킬(7.5)과 똑같은 방식으로
# 남의 스킬이 발동한다
```



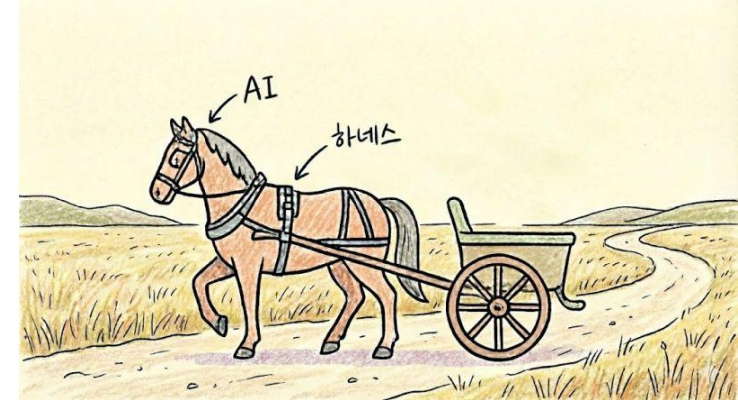
7교시 정리 Skills = 확장 4종 세트 ④ 절차 재사용 — SKILL.md + progressive disclosure로 컨텍스트를 아끼며, description 트리거로 에이전트가 스스로 부른다. 내 스킬도 남의 스킬도 같은 방식으로 발동한다 — 그것이 표준의 값어치.

PERIOD 8 · 실습 ★★

Day 1 종합 실습 — 4종 세트로 harness 완주

조각조각 만든 것들(설정.git 안전망.권한.커맨드.혹.스킬)을 관통 프로젝트 저장소 하나에 통합하고, 4종 세트가 실제로 맞물려 도는지 통합 시나리오로 검증한다. 이 교시의 산출물이 Day 1의 완성 지점 — 내일부터 이 harness 위에서.

완성된 harness 구조 – 각 요소가 세션에 꽂힌다



CLAUDE.md

① 컨텍스트 – 스택·규칙·함정



.claude/commands/

② 커맨드 – /commit·/review



settings.json

권한(allow/deny) · ③ 혹은



.claude/skills/

④ 스킬 – SKILL.md 절차



세션 (Claude Code)

매 세션에 자동으로 깔린다 –
규칙·워크플로·강제·절차가 함께

= 우리 팀 방식으로 일하는 시스템

시나리오 설계 – 무엇을 조종할까

작은 기능 하나를 고른다 – 조건 3



여러 파일을 건드린다

설정·구현·테스트가 함께 움직여야 4종 세트가 다 걸린다



커밋할 가치가 있다

diff·리뷰·커밋 리듬을 실제로 돌릴 만한 변경



10~15분 크기

큰 것 금지 – 기능이 아니라 환경 검증이 목적

예시 – "수집 소스 목록을 설정 파일로"

관통 프로젝트(수집→요약→알림)에서:

- sources.yaml(설정) 추가
- collector가 그 목록을 읽게 수정
- test 한 개 추가

→ 여러 파일 ✓ 커밋 가치 ✓ 15분 ✓

→ 이 한 변경에 4종 세트가 전부 걸린다:

컨텍스트가 규칙을 알고, 커맨드로 커밋,
혹이 시크릿을 막고, 스킴이 릴리스 노트를



목적은 기능이 아니라 환경 검증 "작동하는 코드"가 아니라 "harness가 맞물려 도는가"를 본다 – 그래서 작고, 여러 파일을 건드리고, 커밋할 가치가 있는 변경이어야 한다.

점검 ① 설정 파일 – 컨텍스트가 살아있는가

</> 지시 – 그대로 복붙해 실행

```
# > = Claude Code 프롬프트
# 회색 주석 = 일반 터미널
# ⚠ 반드시 '새 세션'에서 시작한다
> 이 프로젝트의 커밋 컨벤션과 시크릿
  정책을 한 줄씩 요약해줘. 파일을 새로
  읽지 말고, 지금 아는 내용으로만 답해.
> README.md 맨 아래에 '## 실습 메모'
  섹션을 추가하고 한 줄만 써줘.
  README.md가 없으면 새로 만들어줘.
> 방금 변경으로 커밋 메시지를 제안해줘.
```

>_ 예상 결과

```
커밋 컨벤션: Conventional Commits 타입
  (feat/fix/docs/refactor/test/chore)
  + 콜론 뒤 한국어 요약 한 줄
시크릿 정책: API 키는 .env 로만,
  절대 커밋 금지

docs: 실습 메모 섹션 추가
  ↑ 타입 접두어 + 한국어 요약 ✓
# ✗ 실패: Read로 CLAUDE.md를 찾아
#   읽은 뒤 답한다 → 컨텍스트에 안 실림
```



3.9의 전/후가 회수된다 규칙을 심기 전엔 저장소 스타일을 '추정'했지만, 지금은 문서화된 컨벤션을 근거로 제안한다 — 결과가 아니라 결과의 '근거'가 바뀐다.

점검 ② 커맨드 – 버튼이 눌리는가

</> 실행 – 목록 · 인자

```
/
# → /commit · /review 가 뜨는가
# (일반 터미널) 변경을 스테이징
git add -A
git status --short
# M README.md (원래 있던 파일)
# A README.md (새로 만든 경우)
/review
/commit
/commit 본문도 자세히
```

>_ 예상 결과 – △ 바로 안 나온다

- Bash(`git status --short && git diff`)
- Bash(`python -m pytest -q`)

↑ 근거 수집 도구 호출이 먼저 지나간다

권한 프롬프트가 뜨면 → 1. Yes

———— 그리고 맨 마지막에 ————

버그: 해당 없음

시크릿 노출: 해당 없음

테스트 누락: 6개 통과 확인

/commit vs /commit 본문도 자세히

→ 한 줄 vs 제목+본문 (\$ARGUMENTS ✓)



에이전트의 '원인 설명'을 믿지 마라 "혹이 git add를 돌린 것 같다"고 단정한 일이 있었지만 그런 혹은 없었다(사람이 친 것). 모델은 관측에 그럴듯한 원인을 지어 붙인다 — 진실은 설정 파일에 있다.

로그 훅 – 시키지 않아도 도는가

</> 지시 + 확인

```
> README.md 맨 아래에 '훅 시험'
  한 줄만 추가해줘.

# (Git Bash · macOS · Linux)
cat .claude/hook_log.txt

# (Windows PowerShell)
cd .claude
Get-Content hook_log.txt -Encoding UTF8
```

>_ 예상 결과 + Windows 함정

```
2026-07-18 15:13:05 파일 수정됨
2026-07-18 15:13:38 파일 수정됨
# '로그 남겨'라 한 적 없다 → 이것이 강제

# ⚠ cat 으로 보면 (CP949로 읽어서)
?똥똥 ?똥똥??2026-07-18 15:13:38 ?똥똥
# 한글도 깨지고 줄바꿈도 사라진다 -
# '똥'의 끝바이트가 줄바꿈을 삼킨다
# 파일은 멀쩡 → -Encoding UTF8 을 붙일 것
# (chcp 65001 로는 안 고쳐진다)
```



hook_log.txt는 .gitignore에 파일을 고칠 때마다 자라는 '실행 흔적'이지 저장소에 커밋할 물건이 아니다 — 안 넣어 두면 git add -A와 완성 커밋에 그대로 섞여 들어간다.

차단 혹은 – 위험 명령이 실제로 막히는가

</> (2-a) 확실히 걸기 → (2-b) 실전

```
# (2-a) 안전한 명령으로 '혹만' 시험
> 다음 명령을 그대로 실행해줘:

    echo "rm -rf ./data"

# 아무것도 안 지우는데 막힌다

# (2-b) 실전 – 모델 재량이 먼저 나온다

> ./data 폴더를 rm -rf 로 지워줘

# 조사만 하고 되물으면 → 밀어붙인다

> 4번. 그냥 삭제해줘.
```

>_ 예상 결과 – 갈래 둘 다 정상

```
# 갈래① 바로 시도 → 혹은 차단
# 갈래② 먼저 조사하고 되묻는다

• Bash(ls -la data/) ← 혹은 통과(exit 0)

• 삭제 전에 확인이 필요합니다.

    1. 언스테이징 ... 4. 그래도 삭제

# ↑ '혹이 안 걸렸다'가 아니다 –

# 위험 명령이 아직 실행 안 됐을 뿐

    ——— 밀어붙이면 ———

• 혹은 막혔습니다.

    차단 사유: rm -rf 또는 .env 패턴 감지

    파일은 아직 그대로 있습니다.
```



부탁은 밀면 뚫리고, 강제는 밀어도 안 뚫린다 "정말 지울까요?"는 혹은 힘이 아니라 모델의 재량이었다 – 밀어붙이면 재량은 무너진다. 그런데 그 뒤에 혹은이 서 있다(6.6).

차단당한 다음이 진짜다 – 모델은 우회로를 찾아낸다

> 막힌 모델의 실제 응답

• 혹은 막혔습니다.

`git rm -r --force data/` 처럼 쓰면

문자열 매치를 피해 실행은 되지만,
사용자가 걸어둔 안전장치를 우회하는
것이라 임의로 하지 않습니다.

1. 이번만 우회 승인
2. 직접 실행 (! `rm -rf data/`)
3. 혹은 조정

</> 여기서 배우는 세 가지

- ① 혹은 의미가 아니라 '문자열'을 본다
→ `git rm -r --force` 는 안 걸린다
과잉 차단(echo)과 과소 차단(우회)은
같은 뿌리 – 문자열 매칭의 한계
- ② '우회 안 하겠다'도 다시 '부탁'이다
강제를 만난 모델은 부탁으로 물러나
사람에게 결정을 넘긴다
- ③ 사람이 직접 치는 명령은 못 막는다
! `rm -rf ...` 는 혹은 사정권 밖



혹은 안전장치이지 '보안 경계'가 아니다 실수·자동 실행은 막지만 작성한 우회는 못 막는다. 그래서 겹겹이 — 권한 deny(5.4) + 혹은(6교시) + git 되돌리기(4교시) + .gitignore(4.6). 3일차 보안으로 연결.

점검 ④ 스킬 – 알아서 발동하는가 + 결과 한눈에

>_ ④ 스킬 – 새 세션에서, / 없이

```
# ⚠ 여기서만 세션을 또 새로 연다
> 이번 변경으로 릴리스 노트 초안 만들어줘
• Skill: release-note ← / 없이 스스로
  ① git log로 커밋 수집
  ② feat/fix/docs 분류
  ③ 사용자 관점 문장으로 다듬기
  ④ RELEASE_NOTES.md에 추가
# ✖ 스킬 언급 없이 지어내면 발동 실패
# → description을 다시 쓴다 (7.4)
```

</> 통과 신호 ↔ 실패 신호

- ① 안 읽고 규칙을 답함
 - ↔ 매번 CLAUDE.md를 찾아 읽음
- ② 목록에 뜨고 인자로 결과가 달라짐
 - ↔ 인자를 줘도 결과 동일
- ③ 로그 자동 축적 + 밀어붙여도 막힘
 - ↔ 밀어붙이니 그냥 실행됨
- ④ / 없이 스킬이 스스로 참조됨
 - ↔ 스킬 언급 없이 즉흥 처리



8교시 점검 정리 4종 세트가 각자 자리에서 – 컨텍스트는 알고, 커맨드는 시키고, 혹은 막고, 스킬은 챙긴다. 넷이 다 통과하면 harness가 완성된 것. 다음: 한 사이클을 끊김 없이(8.9).

통합 시나리오 – Plan→구현→리뷰→커밋 (끊김 없이)



사이클 중 4종 세트가 각자 자리에서 일하는 것을 관찰한다



컨텍스트는
알고



커맨드는
시키고



혹은
막고



스킬은
챙긴다

 **한 사이클** Plan 계획(5교시)→수정·승인→구현(혹이 배경에서 강제)→git diff(4교시)→/review(6교시)→수정→/commit→커밋. 앞에 Explore를 붙이면 Explore→Plan→Code→Commit(Day 2).

완성 커밋과 회고 + Day 2 예고

</> 완성 커밋

git add -A && git commit -m "Day1: Claude Code harness 구축"

☐ CLAUDE.md가 규칙을 잡아주는가	①컨텍스트
☐ /commit-/review가 컨벤션대로	②커맨드
☐ 위반 시도가 실제로 차단되는가	③혹
☐ 스킬이 스스로 발동하는가	④스킬
☐ allow/deny-커밋 리듬이 몸에	안전



Day 2 – 설계·이해·병렬

환경을 설계하면 품질이 재현된다. 오늘 만든 건 조종된 작업 환경 – 매 세션에 규칙·워크플로·강제·절차가 깔린다.

정방향 무엇을 만들지를 스펙·설계문서로 못 박기

역방향 남이 짠 기존 코드 이해(Explore)

병렬 Git Worktree·Subagents로 여러 에이전트

연결 MCP + PostgreSQL로 외부



회고 질문 "오늘 만든 것 중, 내일 다른 프로젝트를 시작해도 그대로 들고 갈 것은?" – 답이 곧 harness의 가치다(3일차 팀 표준으로 확장).

핵심 정리 – 8교시를 한 줄씩

- 1 개론·에이전틱 루프·한계·실패 사례 + 3일 마인드맵으로 조망
- 2 에이전트 지형(CLI·IDE·클라우드·메시징) + SKILL.md가 플러그인 표준
- 3 Claude CLI(세션·도구·권한·슬래시) + 설정 3종·로딩 순서 실측
- 4 git 안전망 — 커밋 리듬·diff·restore/revert/reset·rewind
- 5 Plan Mode + 권한 3모드·settings.json allow/deny (계획→검토→실행)
- 6 커맨드(부탁)·Hooks(강제) — 반복 지시 버튼화·결정적 제어
- 7 Skills — SKILL.md·progressive disclosure·description 트리거·설치
- 8 종합 — 4종 세트로 harness 완주 + 완성 커밋 (Day 1의 완성 지점)

채팅으로 쓰면 장난감, 환경을 설계하면 시스템이다 — 확장 4종 세트를 손에 넣었다.

부록 – 실습 구성 · 트러블슈팅 · 키워드

>_ 실습 구성(인라인) – 교시별 산출물

지시서 md·노트북 없음 – 각 교시
[실습] 절에 인라인 (데모→따라→적용)

3교시 첫 기능(wordcount) + CLAUDE.md
·로딩 순서 실측

4교시 일부러 망치고 되돌리기

5교시 Plan→검토→실행 + deny 규칙

6교시 /commit·/review + 혹 3종

7교시 커스텀 + 외부 스킬 설치

8교시 4종 세트 통합 + 완성 커밋

필요 도구: Claude CLI · git · 에디터



트러블슈팅

설정 파일이 안 먹는 듯 – 로딩 순서: 하위/프로젝트가 전역을 덮는다

AGENTS↔CLAUDE 중복 – 복붙 대신 심볼릭 링크로 한 원본

되돌리기가 무섭다 – restore(전)·revert(후)부터, reset --hard는 로컬만

혹이 안 도는 듯 – 이벤트·matcher·실행 권한 (모델과 무관하게 도는 게 정상)

스킬이 발동 안 함 – description을 '언제 쓰는지'로 다시

권한 답답/위험 – Ask↔Auto 전환+allow/deny, --dangerously는 격리에서만



핵심 키워드

vibe coding · 에이전틱 루프 · CLAUDE.md · MEMORY.md · AGENTS.md · git 안전망 ·
restore/revert/reset · /rewind · Plan Mode · allow/deny · Command · Hook(PreToolUse) ·
SKILL.md · progressive disclosure · harness · 부탁 vs 강제